Exercise (8.)

(8.1) The majore steps of Decision Tree classification-

Decision Tree construction is a greedy algorithm that construct a decesion-tree.

1. ~~Initiall~~ Tree is constructed in a top-down recursive divide & -and- conquer manner.
2. Initially all the training example are the root.
3. The attributes are considered categorical.
4. The training example are recursively partitioned into smaller subsets based on the selected attributes.
5. Test attributes are selected on the basis of a heuristic or statistical measure.

ex: Information gain.

- Condition for stopping partitioning

1) All samples for a given node belong to the same class.
2) There are no remaining attribute for a given node furthe partitioning - majority voting is employed for classifying the leaf.

3. There are no sample left

(8.2) Tree pruning is a technique in a decision tree conduction, during ~~decesion~~ tree conduction, a decision tree may become unnecessarily large & complex by learning noise or minor variations present in the training data. This can lead to overfitting where the tree performs well on training data but poorly plan on unseen data. Tree pruning remove unnecessary branches.

benefit.

1. Reduce overfitting -
2. Improve generalization -
3. Reduces complexity.
4. Improve interpretability.

Drawback -

1) Several Seperate set of tuples must be reserved for evaluting the pruning process. Consequently fewer training

tuples remain available for constructing the decision tree.

(8.3) In decision tree classification each path from the root node to a leaf node can be converted into an IF-THEN rule. There are two possible approaches to pruning.

approach (a): First convert the decision tree into rules & then prune the resulting rules.

approach (b): First convert the decision tree & then convert the pruned tree into rules.

The main advantage of approach (a) is ~~just~~ that each rules can be pruned independently of the other rules

After converting the tree into rules, unnecessary conditions can be removed from an individual rule without changing the structure of other rules.

for example,

IF Age = Youth AND Student = Yes

AND Income = Medium.

THEN Buys_computer = Yes

If the condition Income = Medium is unnecessary it can be removed.

IF Age = Youth AND Student = Yes

THEN Buys_computer = Yes.

This independent prunning the provides greater flexibility & may produce simpler rules. In contrast, prunning the decesion tree first may affect the multiple paths & the rules generated from those paths.

(8.4) Let, D = The training dataset.
n = The no. of attributes.
|D| = The number of training values.
We need to show that the computational cost of growing a decision tree is atmost

$$n \times |D| \times \log(|D|).$$

Step-1: At each level of decision tree, the algorithm evalutes at most n attributes over the training tuples. Therefore the computational cost, $Cost_{level} = n|D|$.

Step 2: Assuming a balanced binary tree, the no. of tuple is approximately halved at each level.

$$h = \log_2 |D|.$$

Step-3: The total computational cost is

$$Cost = n|D| \times \log_2 |D|.$$

Thus the computational cost of growing the tree is bounded by $O(n|D|\log|D|)$ under the balanced-tree assumption.

(8.5) An efficient method for constructing a decision tree from a large dataset in the RainForest framework, which uses Attribute-Value-Class (AVC) sets to summarize the data.

Step-1: Calculate the no. of AVC entries.

Given, no. of attribute = 50.

Distinct value per " = 100.

no. of classes = 2

AVC entries = $50 \times 100 \times 2 = 10000$

Step2: calculate memory requirement.

Assuming each count requires 4 byte.

Memory = 10000×4.

= 40000 bytes = 40KB.

Step3: Compare with available memory.

RAM = 512 MB = 536,870,912 bytes.

Required = 40000 bytes, therefore the AVC statistics can easily fit into the available main memory.

RAM, OS file.
1 MiB = 1048 byte
= 2^{20} byte

Step4: Tree construction—

Tree construction – The dataset is scanned from disk to generate AVC statistics. These statistics are used to select the best splitting attribute. The dataset is partitioned, & the process is repeated recursively until the decision tree is constructed.

(8.6) Naive ~~Baysiend~~ Baysien classification is a probablistic classification method based on Bayes It is called 'naive' it makes strong assumption that all attributes are conditionally independent of one another, given the class label.

Majore idea of naive Baysian classification

1. Let D be a training set of tuples & their associated class labels, each tuple is represented by an n-D attribute vector.
$X = (x_1, x_2 \cdots x_n)$.
2. Suppose there are m classes $C_1, C_2, C_3 \cdots C_m$
3. Classification is to derive the max posteriori, the maximal $P(C_i|X)$.

4. This can be derive from 'Bayes theorem

$$P(C_i|X) = \frac{P(X|C_i)P(C_i)}{P(X)}.$$

5. Since $P(X)$ is constant for all classes only $P(C_i|X) = P(X|C_i)P(C_i)$. needs to be maximized.

6. Attributes are conditionally independent

$$P(X|C_i) = \prod_{k=1}^{n} P(x_k|C_i) = P(x_1|C_i) \times P(x_2|C_i) \times \ldots \times P(x_n|C_i).$$

These greatly reduce the computational cost. Only counts the class distribution.

7. Class prediction – The posterior probability is calculated for each possible class. The tuple is assigned to the class having the highest posterior probability

(8.7)

(a) The basic decision Tree algorithm should be modified as follows to take into consideration ~~into~~ the count of each generalized data tuple.

1. The count of each tuple must be integrated into the calculation of the ~~electron~~ attribute selection measure,
2. Take the count into consideration to determine the most common class among the tuple.

8.7 (b). Decision Tree from the given info

Here, count = 30+40+40+20+5+3+3+10+4 + 4+6

N = 165

calculate class frequency →.

Senior = 30+5+3+10+4 = 52

Junior = 40+40+20+3+4+6 = 113.

calculate initial Entropy -

Entropy (D) = $-\sum$ Pi log2 Pi

$$= -\frac{52}{165}\log_2\frac{52}{165} - \frac{113}{165}\log_2\frac{113}{165}$$

$(D) = 0.8899$ (original dataset Entropy).

Now, calculate information gain.

(i) Information Gain for Department.

sales – senior = 30

sales – Junior = 40+40 = 80

Total = 110.

systems = 20+5+3+3 = 31. (senior = 8, Jun = 23

marketing = 14, secretary = 10.
$S=10, J=4$

$$\text{Info}_{\text{dept}}(D) = \sum_{j=1}^{v} \frac{|Dj|}{|D|} \times \text{Info}(Dj)$$

$$\text{Info(sales)} = -\frac{30}{110}\log_2\frac{30}{110} - \frac{80}{110}\log_2\frac{80}{110}$$

$$= 0.845$$

$$I_{\text{systems}} = -\frac{8}{31}\log_2\frac{8}{31} - \frac{23}{31}\log_2\frac{23}{31}$$

$$= 0.8244.$$

$$\text{In(marketing)} = -\frac{10}{14}\log_2\frac{10}{14} - \frac{4}{14}\log\frac{4}{14}.$$

$$= 0.8631.$$

Info (secretary) $= -\frac{4}{10}\log_2\frac{4}{10} - \frac{6}{10}\log_2\frac{6}{10}$

$= 0.9710$

Now, Info (Dept) $= \frac{110}{165}(0.8454) + \frac{31}{165}(0.8241)$

$+ \frac{14}{165}(0.8631) + \frac{10}{165}(0.9710)$

$= 0.8504.$

Gain(A) $=$ Info(D) $-$ Info(Dept)

$= 0.8989 - 0.8504.$

Gain(Dept). $= 0.0485.$

(ii)

For Age. 26-30 - senior $= 0$

26-30 - Junior $= 40+3+6$

$= 49.$ Total $= 49$

(A) 21-25 - senior $= 0.$

21-25 - Junior $= 20+$ Total $= 20.$

(B) (31-35) - Senior $= 30+5 = 35$

(C) (31-35) Junior $= 40+4 = 44$ Total $= 79$

(D) (36-40) Senior $= 10$

Junior $= 0$ total $= 10$

(D) (41-45) Senior $= 3.$

Junior $= 0$ Total $= 3.$

(E) (46-50) Senior $= 4$

Junior $= 0$ Total $= 4.$

$$Info(A) = -\frac{0}{20}\log_2\frac{0}{20} - \frac{20}{20}\log_2\frac{20}{20}$$

$$= 0$$

$$Info(B) = -\frac{35}{79}\log_2\frac{35}{79} - \frac{44}{79}\log_2\frac{44}{79} = 0.990$$

$$Info(C) = -\frac{10}{10}\log_2\frac{10}{10} - \frac{0}{10}\log_2\frac{10}{10}$$

$$= 0$$

$$Info(D) = 0.$$

$$Info(E) = 0$$

$$Info(Age) = 0 + 0 + \frac{20}{165} \times 0 + \frac{79}{165} \times 0.990 + \frac{10}{165} \times 0 + 0 + 0$$

$$= \frac{79}{165} \times 0.990 = 0.474$$

$$Gain(Age) = 0.8990 - 0.474$$

$$= 0.4247.$$

(III) Gain (salary).

(A) 26-30k (senior) = 0 junior = 40 46 = 46
Total = 46

(B) 31k-35k (senior) = 0 junior = 40, Total = 40

(C) 36-40k (senior) = 4 junior = 0. Total = 4

(D) 41-45k (S) = 0 junior = 4 Total = 4

(E) 46-50k (S) = 30+10=40 Junior = 23.
Total = 40+23 = 63
(F) ~~51~~ 66-70k (S) = 8 J = 0 Total = 8.

Info(A) = 0
Info(B) = 0
Info(C) = 0.
Info(D) = 0
Info(E) = $-\frac{40}{63}\log_2\frac{40}{63} - \frac{23}{63}\log_2\frac{23}{63}$
= 0.9468
Info(F) = 0.

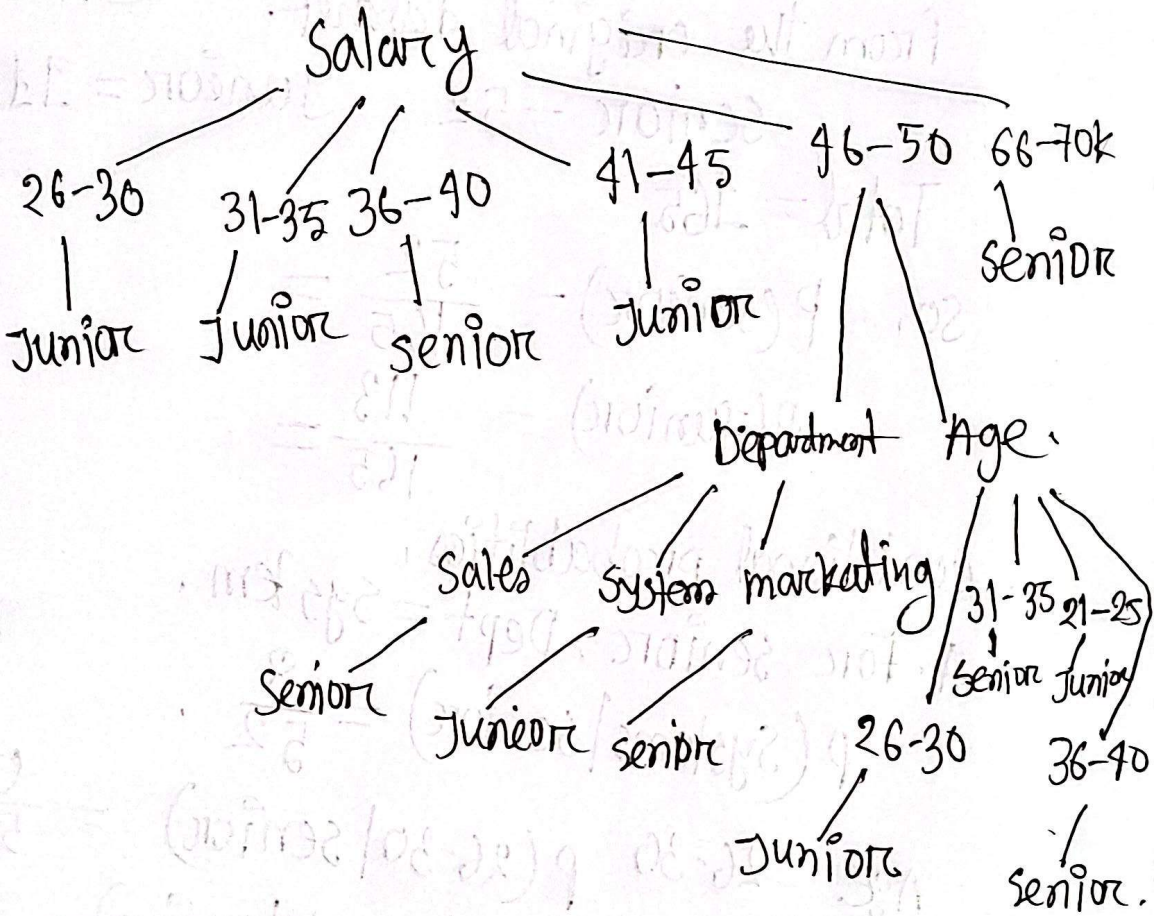
Info(salary) = $\frac{46}{165}\times 0 + \frac{40}{165}\times 0 + 0 + 0 + \frac{63}{165}\times 0.9468 + 0$
= 0.3615.

Gain (salary) = 0.8990 − 0.3615 = 0.5375
From the three attribute gain (salary) is maximum. So, the node of the tree is salary.
salary - 26-30K - senior = 0 Junior = 46 so Decision is Junior. According then - Junior
(31-35) -
36-40k - senior. 41-45K = Junior.
46-50 - sen = 40 Jun = 30 - need spliting
66-70 - S = 8 . J=0 , (senior) .

→ Spliting 46-50k Branch. - Try with the dept.

dept / Age	S	J	
Sales - 31-35	30	0	[46-50k Branch Data].
system - 21-25	0	20	
systems - 26-30	0	3.	
marketing - 36-40	10	0	

Now - Tree -



(e) Given, a data-tuple having:

Department = systems.

Age = 26..30 salary = 46-50k.

Identify the class = senior or junior

Given, tuple is X = (systems, 26-30, 46-50k).

calculate the probablities.

From the original dataset.

senior = 52 junior = 113.

Total = 165

so, $P(\text{senior}) = \frac{52}{165} =$

$P(\text{junior}) = \frac{113}{165} =$

conditional probabilities.

A. For senior. Dept = System.

$P(\text{Systems} | \text{senior}) = \frac{8}{52}$.

Age = 26-30 $P(26\text{-}30 | \text{senior}) = \frac{0}{52}$

salary = 46-50k $P(46\text{-}50k | \text{senior}) = \frac{40}{52}$ → (ওই সেট আইডেন্টিক্যাল rows এর count)

B. For junior:

Dept = system, $P(\text{system} | \text{junior}) = \frac{23}{113}$

Age = 26-30. $P(26|30 \mid \text{Junior}) = \frac{49}{113}$

salary = 46-50k $P(46\text{-}50k \mid \text{Junior}) = \frac{23}{113}$

Naive Bayes ~~For~~

$$P(C|X) \propto P(C) \prod P(x_i|C).$$

A. Senior → $P(X|\text{senior}) = \frac{8}{52} \times \frac{0}{52} \times \frac{40}{52}$

$= 0$

Prior probablity $p(\text{senior}) \times P(X|\text{senior})$

$= 0.$

B. Junior → $P(X|\text{Junior}) = \frac{113}{165} \times$

$= \frac{23}{113} \times \frac{49}{113} \times \frac{23}{113} =$

Prior → Score (Junior) $= \frac{113}{165} \times \frac{23}{113} \times \frac{49}{113} \times \frac{23}{113}$

$= 0.012303.$

score (senior) $= 0.$

Therefore, the given tuple is classified as

status = Junior.

(8.8) → Firstly - A scalable naive Bayesian classification Algorithm can be developed using a single scan of the training dataset

Algorithm - (1) Initialize counters for each class & attribute-value combination.

2. Scan the dataset once - read each tuple exactly once. (3) For each record, update the class frequency & corresponding attribute values frequencies.

3. Calculate prior probabilities - $P(C) = \frac{Count(C)}{N}$

4. Calculate conditional

$$P(A_j = v | C_i) = \frac{Count(A_j = v_j | C_i)}{count(C_i)}.$$

6. Classify new record using,

$$P(C_i|X) \propto P(C_i) \prod_{j=1}^{m} P(X_j|C_i).$$

7. select class - class(x) = arg max$_{C_i}$ $\{P(C_i|X)$

- highest posterior probability.

Boosting → Yes, boosting can be incorporated to improve classification accuracy. By combining multiple weak classifiers and assigning higher weights to misclassified ~~weights~~ records.

However, traditional boosting requires multiple training rounds & often repeated dataset scans. Therefore, it conflicts with the strict-single scan requirement. Approximate or online boosting techniques may be used. but they introduce additional complexity.

(89) An efficient Naive Bayesian classifier for an infinite data stream can be designed using incremental learning and online updating of sufficient statistics.

step1 :- Initialize counters for class frequencies and attributes-value frequencies.

2. Process each incoming record only once

3. Update the corresponding counters whenever new record arrives.

4. Calculate prior & conditional probabilities using the statistics.

5. Classify new incoming records using the updated probabilities.

6. Store only sufficient statistics instead of storing the entire data stream.

Discover the classification schemes —

- To discover the evalution of classification schemes over time, a time-aware or historical version of the incremental Naive Bayesian classifier should be maintained.

The system should periodically save snapshots of its sufficient statistics & model parameters. For example, solve the model at the end of the day or week. These historical snapshots can be compared with the current model to identify the changes in classification behavior over time.

(8.10) We know that, accuracy = $\frac{TP+TN}{P+N}$.

Here, TP = True positive,
TN = True Negative.
P = Total actual positive tuples.
N = Total actual negative tuples.

equation → semetivity

Semetivity = $\frac{TP}{P}$.

TP = Semtivity × P.

equation Specificity, $= \frac{TN}{N}$.

$TN =$ seceficity $\times N$.

$$\text{Accuracy} = \frac{TP + TN}{P+N}$$

$$= \frac{\text{semtivity} \times P + \text{Specificity} \times N}{P+N}$$

By seperating.

$$\text{Accuracy} = \frac{\text{Semitivity} \times P}{P+N} + \frac{\text{specificity} \times N}{P+N}$$

$$\Rightarrow \text{Accuracy} = \text{semitivity} \frac{P}{P+N} + \text{specificity} \frac{N}{P+N}$$

Hence Accuracy is weighted average of semetivity & specificity, where the weight are the proportion of positive & negative tuples, respectively

(8.11) The Harmonee mean of positive real numbers $x_1, x_2, \ldots x_n$ is defined as.

$$H = \frac{n}{\sum_{i=1}^{n} \frac{1}{x_i}}$$

For two numbers $H = \frac{2}{\frac{1}{x_1} + \frac{1}{x_2}}$.

Derive, F-measures.

We know, F-measure is the harmonic mean of precision & Recall.

Let, P_r = Precision R = recall

Using harmonic mean, $F = \frac{2}{\frac{1}{P_r} + \frac{1}{R}}$.

$$\Rightarrow F = \frac{2}{\frac{R+P}{P_r \times R}}$$

Therefore $F = 2 \times \frac{P_r \times R}{R + P_r}$

$$F = \frac{2 P_r \times R}{P_r + R}.$$

Express F in term of TP, FP & FN.

Precision, $P_r = \frac{TP}{TP + FP}$

Recall $R = \frac{TP}{TP + FN}$.

Substituting these into the F-measure formula

$$F = \frac{2 \times \frac{TP}{TP+FP} \times \frac{TP}{TP+FN}}{\frac{TP}{TP+FP} + \frac{TP}{TP+FN}}$$

$$\Rightarrow F = \frac{2TP^2}{TP(TP+FN) + TP(TP+FP)}.$$

$$\Rightarrow F = \frac{2TP^2}{2TP(TP+FN)+TP^2+TPFN+TP^2+TPFP}$$

$$= \frac{2TP^2}{2TP^2+TP(FN+FP)}$$

$$= \frac{2TP^2}{TP(TP+FN+TP+FP)} = \frac{2TP}{2TP+FN+FP}$$

$$f = \frac{2TP}{2TP+FP+FN}.$$

→ Derive FB-measure.

$$FB = \frac{(1+B)^2 P_r R}{B^2 P_r + R}.$$

substitute, $P_r = \frac{TP}{TP+FP}$, $R = \frac{TP}{TP+FN}$.

$$F_B = \frac{(1+B)^2 \frac{TP}{TP+FP} \frac{TP}{TP+FN}}{B^2 \frac{TP}{TP+FP} + \frac{TP}{TP+FN}}.$$

Simplifying, $F_B = \frac{(1+B)^2 TP^2}{B^2 TP(TP+FN)+TP(TP+FP)}.$

$$F_\beta = \frac{(1+\beta)^2 TP}{\beta^2(TP+FN)+(TP+FP)}$$

$$Exp = \frac{(1+\beta)^2 TP}{\beta^2 TP + \beta^2 FN + TP + FP}$$

$$= \frac{(1+\beta)^2 TP}{TP(1+\beta^2) + \beta^2 FN + FP}$$

(8.12) Given, Tuple class probability
1 p 0.95.

Total p = 5, To N = 5. Tuples = 10.

True positive Rate, $TPR = \frac{TP}{TP+FN}$

False positive Rate, $FPR = \frac{FP}{FP+TN}$

[TP = Actual p, Predicted p.
FP = Actual N, Predicted p.
TN = Actual N, " N.
TP = Actual P, " N.

Initially, TP = 0 FP = 0. TN = 5, FN = 5.

Now for every Tuple.

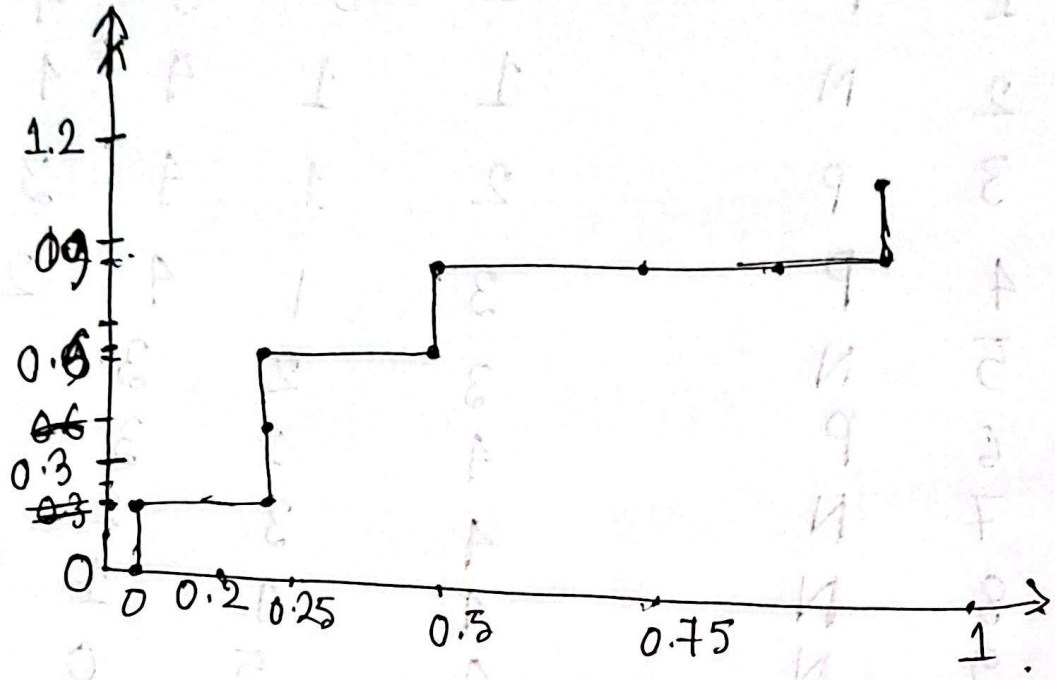
Tuples	Actual	TP	FP	TN	FN
Initial		0	0	5	5
1	P	1	0	0	4
2	N	1	1	4	4
3	P	2	1	4	3
4	P	3	1	4	2
5	N	3	2	3	2
6	P	4	2	3	1
7	N	4	3	2	1
8	N	4	4	1	1
9	N	4	5	0	1
10.	P	5	5	0	0

[→ Actual positive - TP +1 & FN-1.
Actual N - FP+1 TN-1]

Now. TPR & FPR = $\frac{FP}{5}$.

Tuples	$\frac{TP}{5}$.	TPR	FPR	Tuples	TPR	FPR
1	1	0.2	0.0	10	1.0	1.0
2	2	0.2	0.2			
3	3	0.4	0.2			
4	4	0.6	0.2			
5	5	0.6	0.4			
6	6	0.8	0.4			
7		0.8	0.6			
8		0.8	0.8			
9		0.8	1.0			

ROC curve — X-axes = False positive Rate (FPR)
Y-axes = True positive Rate



(8.13) When individual data objects belong to multiple classes simultaneously, traditional classification accuracy is not sufficient for evaluating classifier performance.

1. Precision - ~~Predicts~~ Measures how many predicted class labels are actually correct.

$$\text{Precision} = \frac{TP}{TP + FP}$$

2. Recall (sensitivity) - Measures how many actual class labels are correctly identified

$$\text{Recall} = \frac{TP}{TP + FN}$$

3. F1-score - combines precision & recall using their harmonic mean

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}.$$

4. Hamming loss -
measure the fraction of incorrectly predicted class labels among all possible-object-class pairs. Hamming loss = $\frac{\text{Incorrected labels}}{\text{Total possible labels}}$.
lower hamming loss indicates better performance

5. Computational Efficiency → compare the time & computational resource required to train & use each classifier.

(8.14): Here. Given 10-fold cross-validation
The error rate M1 & M2, differen

$$t = \frac{\overline{err}(M_1) - \overline{err}(M_2)}{\sqrt{var(M_1 - M_2)/k}}$$

$$var(M_1 - M_2) = \frac{1}{k} \sum_{i=1}^{k} [err(M_1)_i - err(M_2)_i - (\overline{err}(M_1) - \overline{err}(M_2))]^2$$

($M_1 - M_2$ = Given)

$d_i = M1_i - M2_i$

Round	d_i	$(M1 - M2)^2$ $(d - 6.45)^2$
1	8.1 (−64)	$(8.1 - 6.45)^2$ 2.7225
2	17.7	126.5625
3	−1.7	64.4125
4	1.0	29.7025
5	10.3	14.82225
6	20.6	0.7225
7	5.6	0.0225
8	6.6	1.3225
9	5.3	
10	−9.0	238.7025

$\Sigma d_i = 64.5$

$$Sd = \sqrt{\frac{\Sigma(d_i - \bar{d})^2}{n-1}}$$

$$\Sigma d_i = \frac{\Sigma d_i}{n} = \frac{64.5}{10}$$

$= 6.45$

$$\Sigma(d_i - \bar{d})^2 = 681.225$$

$n = 10$

$$\text{So, } S_d = \sqrt{\frac{681.225}{10-1}}$$

$$S_d = \sqrt{75.6917}$$

$$S_d = 8.7001$$

$$\text{Now, } t = \frac{d_i}{S_d/\sqrt{n}} = \frac{6.45}{8.7001/\sqrt{10}}$$

$$= \frac{6.45}{2.7519}$$

$$t = 2.344$$

at 1% significance level. $t_{critical} = 3.250$

& dof = 10 - 1 = 9

Hypothesis: H_0: There is no significant difference between M1 & M2.

H_1: There is a significant difference between M1 & M2.

Rule: If $|t| > t_{critical}$, reject H_0

If $|t| < t_{critical}$, fail to reject H_0

Here $2.344 < 3.250$.

There we will fail to reject H_0 which model better.

$$\text{Mean}(M1) = \frac{277.2}{10} = 27.72$$

$$\text{Mean}(M2) = \frac{212.7}{10} = 21.27$$

Although M2 has lower average rate than M1 the difference is not statistically significant at the 1% significance level. We can't conclude that the either model is significantly better than other.

(6.15)

Boosting: Boosting is an ensamble learning technique in which a sequence of classifiers is constructed, and each subsequent classifier focuses more on the training tuples misclassified by the previous classifiers.

working procedure:

1. Train an initial weak classifier using the training dataset.
2. Identify the incorrectly classified records
3. Assign hegher weights to misclassified records

4. Train another classifier that focuses more on the difficult records.
5. Repeat the process for several iteration
6. Combine the predictions of all classifier using weighted voting.

Boosting improves Decision Tree Accuracy.

1) It focuses on previously misclassified instances.
2) It reduces classification errors.
3) It combine multiple weak decesion trees into a stronger classifier.
4) It capture complex patterns in the dataset.
5) It can improve the overall predective accuracy.

(8.16) Class imbalance occurs when the number of instances in one class is significantly larger than the number of instances in another class.
for example 99% In credit card fraud detection, 99% of transactions may be non-fraudlent, while only 1% are fraudlent.

Method for Addressing class imbalance—

1. Undersampling: Reduce the no. of majority class instance.
2. Oversampling: Increase the no. of minority-class instances by duplicating existing samples.
3. SMOTE: Generate Synthetic minority-class samples
4. Cost-sensetive learning: Assign higher misclassification costs to the minority class.
5. Ensemble learning: Combine multiple classifiers to improve minority-class detection.

To develop a Quality Fraud Detection classifier.

Step 1: Collect training Data.

Step-2: Identify minority & majority classes.

- Non-fraudlent = majority class.
  Fraudlent - Minority classes.

3) Apply oversampling or SMOTE.

4) Apply undersampling if necessary.

5) Train an Ensemble classifier.

6) Adjust the classification Threshold - select a suitable threshold to improve fraud detection while controlling false alarm.

Step-7:- Evalute the classifier.

using - Sensetivity (Recall).
- Precesion.
- F1-score.
- Specificity.
- ROC-AUC.

